Codebase Change Summary — Week 8

Functional Testing

SE 413/513 — Software Testing

Demo Testing Suite

What changed in `demo-testing-suite/` since Week 7, and why

1 Purpose of This Document

The `demo-testing-suite/` codebase is a single, shared project that grows one lecture at a time rather than being rebuilt from scratch each week. This document summarizes exactly what was added or changed for Week 8 and, more importantly, explains *why* each change exists — which concept from the Week 8 reading, *Functional Testing: Designing Tests from the Specification*, it demonstrates. Read this alongside the reading and `demo-testing-suite/README.md`.

Key Idea

Nothing from Weeks 1–7 was removed or changed. This week adds one small, self-contained project that runs every example in the reading: the decision table for shipping fees, the `Order` state machine, the pairwise checkout rows, and the Hypothesis property for splitting a bill — together with the buggy versions each technique catches. The main suite is untouched, so its expected results are the same as last week.

2 What Changed At a Glance

File	Status	Purpose
The reading, runnable (not in the default run): `demos/functional_testing/`		
`shop/shipping.py`, `shop/shipping_buggy.py`	New	`shipping_fee` for REQ-1–4, and a version that checks the free-shipping rule first.
`shop/order.py`	New	The `Order` state machine for REQ-5–7.
`shop/split.py`, `shop/split_fixed.py`	New	`split_bill` for REQ-8: the reading's buggy first version, and the fix in integer cents.
`tests/test_shipping_dt.py`	New	One test per decision-table rule (R1–R5).
`tests/test_shipping_bva.py`	New	The $50 boundary: 49.99 and 50.0.
`tests/test_order.py`	New	All-transitions coverage: 5 valid, 15 invalid, 2 refund tests.
`pairwise.py`, `tests/test_checkout_pairwise.py`	New	The `allpairspy` rows, a check that every pair is covered, and the rows as `pytest` cases.
`tests/test_split_pbt.py`	New	The Hypothesis property for REQ-8.
`README.md`	New	Every command, in the reading's order, with its expected result.
Tooling and documentation		
`requirements.txt`	Modified	Adds `hypothesis` and `allpairspy`.
`.gitignore`	Modified	Ignores `.hypothesis/`, Hypothesis's local example database.
`docs/DEMO_GUIDE_week08_functional_testing.md`	New	Section-by-section lecture companion for the reading.
`README.md` (suite and repository)	Modified	Updated layout and the Weekly Updates index.

Tip

Reinstall the requirements before running anything this week:

```
cd demo-testing-suite
pip install -r requirements.txt
```

3 The Reading's Examples, Runnable

Location: `demos/functional_testing/`

This folder is laid out exactly like the reading (`shop/shipping.py`, `tests/test_order.py`, ...), so the reading's `from shop.`*module* `import` lines and `pytest` commands work as printed. **Run its commands from inside the folder:**

```
cd demos/functional_testing
```

Run	Result	Reading section
`pytest tests/test_shipping_bva.py`	2 passed	Recap: EP and BVA
… after changing `>= 50` to `> 50`	fails at `50.0`	Recap: EP and BVA
`pytest tests/test_shipping_dt.py -v`	5 passed, IDs R1–R5	One Test per Rule
… importing `shop.shipping_buggy`	R1 fails; 1 failed, 4 passed	What It Catches
`pytest tests/test_order.py`	22 passed	Implementing the Tests
`python pairwise.py`	10 rows; 45 of 45 pairs	The Pairwise Idea
`pytest tests/test_checkout_pairwise.py`	10 skipped (no oracle yet)	Limitations
`pytest tests/test_split_pbt.py`	fails; shrunk to `total_cents=1, people=2`	From Examples to Properties
… importing `shop.split_fixed`	1 passed	Fixing the Defect

Common Pitfall

Two demos are run by changing one import line, and they go in opposite directions. `tests/test_shipping_dt.py` imports the *correct* `shop.shipping`; you switch it to `shop.shipping_buggy`. `tests/test_split_pbt.py` imports the *buggy* `shop.split`; you switch it to `shop.split_fixed`. That is why a plain `pytest` in this folder reports `1 failed, 29 passed, 10 skipped`: the one failure is REQ-8's, on purpose. Switch any import you changed back afterwards.

Why the pairwise test is skipped

The reading leaves the body of `test_checkout_pairwise` as an ellipsis (`...`), a placeholder. Run that way, all ten rows would *pass* while checking nothing — Week 7's "100% coverage, zero assertions" trap in a new form. Pairwise testing chooses combinations; it does not supply expected results. Until there is a `checkout()` function and an expected result for each row, every row is skipped with the reason "no oracle yet".

`.hypothesis/`

Hypothesis saves every failing input in a local database (`.hypothesis/`) and tries those inputs first on the next run, so a bug that comes back is caught at once. Delete the folder to watch Hypothesis search from scratch. It is listed in `.gitignore`.

4 Something to Check Yourself

The pairwise slide's "Done early?" asks whether every valid pair is still covered after filtering out "gift card with a non-USD currency". Run the same coverage check that `pairwise.py` uses on the filtered set before you answer.

Key Idea

A generated test set is a claim, not a guarantee. Last week, a coverage number needed reading before it could be trusted; this week, the output of a pairwise generator does. A few lines of checking code — like the one in `pairwise.py` — tell you whether the claim holds.

5 Updated: Expected Test Results

From the project root:

```
cd demo-testing-suite
pip install -r requirements.txt
pytest -v
```

you should still see **72 passed, 2 xfailed, 1 failed**, exactly as in Week 7. `demos/functional_testing/` is not part of that run (`pytest.ini` sets `testpaths = tests`); run it from inside its folder.

6 How This Connects to the Reading

Key Idea

Weeks 5–7 asked whether the tests exercised the code. This week asks whether the code does what the specification says. Each file in the demo folder is one model from the reading — a table, a state machine, a parameter set, a property — turned into tests, and each buggy version is a defect that the model finds and a coverage report would not.

7 Suggested Next Steps

1. Switch `tests/test_shipping_dt.py` to the buggy version and add a case for rule R2 at $80. Predict the result before running it. Then switch the import back.
2. Add the `return_item` event to `shop/order.py` (Exercise 3) and extend `tests/test_order.py` to keep all-transitions coverage. How many tests pass now?
3. Add `currency` to `pairwise.py` (Exercise 4), filter out gift card with a non-USD currency, and run the coverage check on the filtered rows.
4. Write the two `apply_coupon` properties from Exercise 5 in `demos/coverage_reports/`. If Hypothesis finds a counterexample, decide whether the defect is in the code or in the property.
5. Run `pytest --cov=shop --cov-branch --cov-report=term-missing` on just the two shipping test files, and explain why four `shop/` modules report 0%.